

AptitudeMind

Complete Development Record & Project Progress Documentation

Adaptive Agentic AI Mentor

Purpose: This document preserves the AptitudeMind project information accumulated during development. It is a development record, not merely a short abstract. Completed, partial, planned, tested and debugging information are retained.

Preservation rule: Future development should add information to this record rather than silently removing earlier milestones, tests, bugs, fixes or design decisions.

1. Project Identity

Project: AptitudeMind — An Adaptive Agentic AI Mentor.

Core idea: Build an AI aptitude mentor that understands the learner's topic, difficulty, previous performance and weak areas; selects or generates a question; validates it; evaluates the learner's answer; remembers the result; and adapts future practice.

Development philosophy: local-first, hands-on learning, understanding before copy-paste, incremental implementation, testing after major components, and preserving the full project vision despite deadline pressure.

2. Original Full Vision

- LLM fundamentals
- Tools / function calling
- Prompt engineering
- Agent workflow and decision making
- Memory
- RAG / knowledge retrieval
- Multi-step / autonomous tasks
- Error handling and guardrails
- Testing and evaluation
- API + deployment
- Clean UI
- Documentation
- GitHub project and final presentation

Full application vision: Quantitative Aptitude, Logical Reasoning and Verbal Ability; AI-generated and verified questions; adaptive difficulty; persistent/topic performance memory; weak-topic analysis; RAG; agent decision workflow; testing/evaluation; API/backend; student dashboard; practice arena; AI mentor chat; topic library; analytics/progress; gamification; and deployment.

3. Intended Adaptive Flow

```
Student
↓
Website
↓
AptitudeMind Agent
↓
Topic selection
```

↓
AI question generation / retrieval
↓
Python verification
↓
Student answer
↓
Evaluation
↓
Memory
↓
Weak-topic detection
↓
Adaptive next question

Core architecture: User → Agent → LLM + Tools + Memory → answer. Later layers add calculator, question database, progress tracking, RAG and adaptive/autonomous workflow.

4. Environment and Setup

- Ubuntu
- VS Code preferred
- Python 3.12.3
- Ollama 0.33.3
- llama3.2:3b local model
- ~/AptitudeMind
- Python .venv
- Git/GitHub

The project uses a local-first approach because the user does not have a budget for paid AI APIs.

5. Important Project Files

- main.py — large original/full terminal application
 - agent.py — agent decision and execution layer
 - generator.py — Ollama generation
 - validator.py — question validation
 - aptitude_tools.py — deterministic calculations
 - memory.py — persistent memory
 - progress.py — progress exports
 - difficulty.py — adaptive difficulty
 - topics.py — aptitude taxonomy
 - companies.py — company metadata
 - search.py — query parsing
 - search_engine.py — retrieval integration
 - question_bank.py — sample question bank
 - progress.json — runtime progress
- .gitignore includes .venv/, .env, __pycache__/, *.pyc and progress.json.

6. Calculator / Tools

```
def calculate_percentage(value, percentage):  
    return value * percentage / 100
```

```
def calculate_profit(cost_price, profit_percentage):
    profit = calculate_percentage(cost_price, profit_percentage)
    return cost_price + profit

def calculate_loss(cost_price, loss_percentage):
    loss = calculate_percentage(cost_price, loss_percentage)
    return cost_price - loss

def calculate_average(numbers):
    return sum(numbers) / len(numbers)

def calculate_simple_interest(principal, rate, time):
    return principal * rate * time / 100

def calculate_distance(speed, time):
    return speed * time

def calculate_speed(distance, time):
    return distance / time

def calculate_time(distance, speed):
    return distance / speed
```

The deterministic tool layer reduces reliance on the LLM for numerical calculation.

7. Memory and Progress

AptitudeMind stores total attempts, correct answers, wrong answers and topic-level performance. Persistent progress is stored in progress.json so terminal clearing does not erase it.

```
{
  "total": 0,
  "correct": 0,
  "wrong": 0,
  "topics": {}
}
```

Topic records contain attempted, correct and wrong counts. Overall accuracy is $\text{correct} / \text{total} \times 100$. Topic accuracy is calculated similarly. Weak-topic detection selects the topic with the lowest observed accuracy.

The user explicitly requested that progress.json not be deleted during this development/documentation phase.

8. Adaptive Difficulty

Easy / Medium / Hard

Accuracy < 50% → Easy
 50% to 75% → Medium
 Accuracy > 75% → Hard

Exactly 50% is Medium. A topic with no attempts starts at Easy. The difficulty layer was tested successfully.

9. Aptitude Topic System

Quantitative Aptitude

Percentage; Profit and Loss; Ratio and Proportion; Average; Number System; Simple Interest; Compound Interest; Time and Work; Pipes and Cisterns; Time Speed and Distance; Problems on Trains; Boats and Streams; Ages; Mixtures and Allegations; Partnership; Permutation and Combination; Probability; Algebra; Progressions; Geometry; Mensuration; Data Interpretation; Data Sufficiency.

Logical Reasoning

Number Series; Letter Series; Coding and Decoding; Analogy; Classification; Blood Relations; Direction Sense; Syllogism; Statements and Conclusions; Statements and Assumptions; Seating Arrangement; Puzzles; Venn Diagrams; Clocks; Calendars; Logical Sequence; Pattern Recognition.

Verbal Ability

Synonyms; Antonyms; Vocabulary; Idioms and Phrases; Fill in the Blanks; Sentence Completion; Error Spotting; Sentence Correction; Grammar; Active and Passive Voice; Direct and Indirect Speech; Para Jumbles; Reading Comprehension;

Sentence Arrangement; Word Usage.

10. Company System

Companies currently represented: TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini, Deloitte, IBM, HCL, Tech Mahindra, Amazon, Microsoft, Google, Goldman Sachs, JPMorgan Chase, Oracle, Cisco and Walmart.

Company categories include IT Services, Consulting, Technology, Product Company, Finance / Technology and Retail / Technology.

Question-bank labels such as company_style mean company-style practice content, not automatically verified previous-year company questions.

11. Search System

Search normalizes user text and detects company, topic and difficulty. The search engine connects parsed filters to the question bank.

A bug was found where an unknown query such as 'ff' could return all questions because empty filters matched everything. The agent was changed to check whether any meaningful filter was detected and ask for a clearer request otherwise.

Tests included TCS percentage medium, Amazon probability hard, hard time-speed-distance and Infosys reasoning.

12. Question Bank

The bank contains sample company-style questions across topics and difficulty levels. Helpers include get_all_questions(), search_questions(company, topic, difficulty) and get_question(company, topic, difficulty).

Sample coverage includes TCS Percentage, Infosys Profit and Loss, Wipro Average/Synonym/Antonym, Accenture Ratio, Cognizant Time and Work, Amazon Probability, Microsoft Number System, Deloitte Number Series, IBM Coding and Capgemini Direction.

13. AI Generator

```
import ollama

def generate_question(topic, difficulty, company=None):
    company_text = company if company else "general placement"
    prompt = f'''
You are an expert aptitude question generator.
Create ONE {difficulty}-level aptitude question.
Topic: {topic}
Company style: {company_text}
Requirements:
1. Clear and unambiguous.
2. Exactly one correct numerical or textual answer.
3. Do not provide the answer.
4. Do not provide explanation.
5. Do not create a trick question.
6. Return only the question.
Question:
'''
    response = ollama.chat(
        model="llama3.2:3b",
        messages=[{"role": "user", "content": prompt}]
    )
    return response["message"]["content"].strip()
```

The generator uses the local llama3.2:3b model and requests a single question without answer/explanation.

14. Agent Decision Layer

The agent decides among retrieve, generate and ask. If a topic is supplied without difficulty, adaptive difficulty is selected from stored performance. Free-form requests can include company, topic and difficulty.

15. Agent Execution Layer

Retrieved questions are validated before use. Generated questions are structured with question, topic, difficulty, company and source. Generation is attempted up to three times; failed validation causes regeneration. After three failures the system safely returns an error instead of recording an untrusted question.

16. Validator / Guardrails

Validator checks structure, question presence, topic, difficulty, company, options, minimum length and ambiguous wording.

Flagged ambiguous phrases include 'one of the roots', 'one of the answers', 'any value', 'could be' and 'might be'.

Percentage questions are checked for percentage context and discount wording. Algebra questions receive mathematical consistency checks.

17. Algebra Validation Upgrade

An early Ollama-generated question used $x^2 - 5x - 6 = 0$ and incorrectly described the sum/product of roots, while also asking for 'one of the roots'. The validator was designed to reject this ambiguity and mathematical inconsistency.

Vieta checks compare the claimed sum with $-b/a$ and product with c/a .

A second generated Algebra question claimed that $3x^2 + 5x - 8 = 0$ had discriminant 16. The actual discriminant is 121. Discriminant validation was added using $\Delta = b^2 - 4ac$.

18. Validator Test Results

Test	Purpose	Result
1	Valid Percentage question	VALID
2	Missing/invalid data	INVALID; expected errors
3	Ambiguous Algebra	INVALID
4	Incorrect sum of roots	INVALID
5	Incorrect discriminant	INVALID

Test 5 output: Mathematical inconsistency: discriminant should be 121, not 16.

19. Main Application

main.py contains Ollama integration, difficulty selection, calculator tools, memory, topics, response parsing, robust validation, deterministic answer calculation, explanations and mistake analysis.

The main application includes semantic/type consistency guardrails. A discount question was once mislabeled as generic Percentage, causing the correct final price to be evaluated incorrectly; this led to stronger discount/final-price and increase wording checks.

Main generation uses a three-attempt generation/validation loop and does not record a question after all attempts fail.

Mistake analysis includes Percentage/INCREASE cases where a learner may enter an intermediate increase amount rather than the requested final value.

20. Recorded Progress Snapshot

Recent known snapshot: 16 total attempts, 6 correct, 10 wrong; overall accuracy 37.5%; Percentage 6/16. This is a development snapshot, not a final project result.

21. GitHub Milestones

GitHub repository: [KarthikRP424/AptitudeMind](#).

A documented checkpoint used commit 79c5331 with message 'Improve adaptive question generation and validation', involving agent.py, generator.py, nano difficulty.py, nano validator.py, main.py and search_engine.py.

The validator file was later renamed from the accidental filename 'nano validator.py' to validator.py, and 'python -c "from validator import validate_question; print('\Validator import: OK\')"' succeeded.

The agent/validator checkpoint was uploaded and the user confirmed it was already on GitHub.

22. Documentation History

Earlier project documentation artifacts included AptitudeMind_Professional_Documentation_v1.docx and AptitudeMind_Recent_Development_Progress.pdf. This new PDF is intended as the broader complete development record and does not replace the historical information in those records.

23. Current Overall Progress

Estimated overall completion toward the complete long-term vision: approximately 65%. This is a development estimate, not a formal engineering metric.

- Ollama/local LLM — 100%
- Calculator/tools — 100%
- Persistent memory — 100%
- Topic system — ~90%
- Company system — ~90%
- Question bank/search — ~85%
- Adaptive difficulty — ~90%
- Agent decision layer — ~85%
- Agent execution layer — ~80%
- Validator/guardrails — ~70%
- Self-correction/regeneration — ~70%
- RAG — ~0%
- Complete agent workflow — ~50%
- Website/UI — ~0%
- API/backend — ~0%
- Testing/evaluation — ~30%
- Deployment — ~0%
- Final documentation/demo — ~40%

24. Remaining Roadmap

- Finish validator coverage and agent integration
- Verify generation → validation → regeneration loop
- Build RAG / aptitude knowledge retrieval
- Complete multi-step adaptive agent workflow
- Integrate question selection, generation, validation, answer evaluation and memory
- Build backend/API
- Build website/application interface
- Student dashboard
- Practice arena

- AI mentor chat
- Topic library and analytics
- Testing/evaluation and edge cases
- Deployment
- Final documentation, demo and presentation

25. Final Demo Concept

```

Student
↓
Select domain / topic / company
↓
Agent checks memory
↓
Adaptive difficulty
↓
Retrieve or generate
↓
Validator
↓
Invalid → regenerate
↓
Valid → answer
↓
Verified evaluation
↓
Explanation / mistake analysis
↓
Progress memory
↓
Weak-topic detection
↓
Adaptive next question

```

26. Engineering Principles

- Do not remove the full vision because of deadline pressure.
- Use deadlines to prioritize a demo-ready order, not to delete capabilities.
- Use deterministic Python verification for mathematical correctness.
- Treat the LLM as a generator/reasoning component, not an unquestioned source of truth.
- Validate generated questions before presenting them.
- Persist learning history for adaptation.
- Build → test → debug → connect.
- Preserve complete development history for final documentation.
- Learn the architecture instead of blindly copying an agent.

27. Latest Development Position

The latest completed step is the validator upgrade with discriminant validation. All five validator tests produced the expected outcomes. The immediate next action is to run the Agent with a request such as 'TCS Algebra Hard' and verify that the upgraded validator is used by the agent's self-correction loop.

No further code change is required before that integration test.

28. Closing Record

AptitudeMind is being built as both a learning project and portfolio project. Its implementation demonstrates Python, local LLM usage, prompt engineering, tools, memory, adaptive logic, validation and agent workflow, with RAG, API, UI, evaluation and deployment planned as the next major layers.

This record intentionally preserves successful implementations, bugs, failed generations, debugging decisions and fixes because those details demonstrate how reliability is being engineered into the agent.